

```html



# Everything About GSAP

A Comprehensive Guide to GreenSock Animation Platform



## 1. Introduction to GSAP

- **GSAP (GreenSock Animation Platform):** A powerful JavaScript animation library.
- **Purpose:** To perform robust animations on web pages, including scroll triggers and SVG animations.
- **Series Outline:**
  - GSAP Basics 🎯
  - ScrollTrigger Integration 📄
  - SVG Animation Techniques 🖼️
  - Five Practical Projects 💡
- **Animation Schedule:**
  - GSAP videos: Mornings (around 11 AM - 12 PM) ☀️
  - Backend videos: Evenings (around 8 PM) 🌙
- **What are Animations?** ✨
  - *Movement, transitions, or visual effects on web pages that enhance user engagement and experience.*
  - Examples: Hover effects on buttons, moving elements.
  - Goal: Make websites more attractive and interactive.
- **What are Libraries?** 📦
  - *Pre-written code or a collection of features designed to perform specific tasks.*
  - Benefits: Simplify complex development, provide specific functionalities.
  - Examples of other JavaScript libraries:
    - **Locomotive.js:** For smooth scrolling effects. 📄
    - **React:** For building entire user interfaces (UI) and front ends. ⚙️
    - **Swiper.js:** For creating sliding animations (carousels, sliders). ➡️
    - **GSAP:** Specifically for moving animations, transitions, and effects related to movement. 🔄



## 2. Setting Up GSAP

```
+-----+
| GSAP Folder |
+-----+
| - index.html |
| - style.css |
| - script.js |
+-----+
```

- **Prerequisites:** Basic HTML and CSS knowledge is sufficient. JavaScript is not strictly required to follow along, as the GSAP syntax will be explained.
- **Folder Structure:**
  - Create a folder (e.g., 'GSAP').
  - Inside, create 'index.html', 'style.css', and 'script.js'.
- **HTML Boilerplate:** Use a standard HTML5 boilerplate in 'index.html'.
- **Linking Files:**
  - Link 'style.css' in the `` of 'index.html'.
  - Link 'script.js' at the end of the `` in 'index.html'.
- **Including GSAP via CDN:**
  - *GSAP's core code resides on a cloud server; a CDN (Content Delivery Network) link creates a connection to access its functionalities.*
  - Search: "GSAP CDN" on Google. 🔍
  - Copy the script tag from the official GSAP CDN (usually the second option for "Copy Script Tag").
  - Paste this GSAP script tag in 'index.html' \*just above\* your 'script.js' link.
  - ```
<!-- GSAP CDN link (comes first) -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js" integrity="..." crossorigin="anonym
<!-- Your custom JavaScript (comes second) -->
<script src="script.js"></script>
```



3. Basic GSAP Usage: `gsap.to()` and `gsap.from()`

```
(Initial Position) --> (Final Position) = gsap.to()
(Final Position) <-- (Initial Position) = gsap.from()
```

- **Core Syntax:** GSAP animations are written using `gsap.to()`, `gsap.from()`, or `gsap.fromTo()`.
- **Structure:** `gsap.method(target, vars)`
 - **Target:** *The HTML element(s) you want to animate.*

- Use standard CSS selectors (e.g., `"#box"`, `".element"`, `"h1"`).
 - Place selectors inside double quotes.
- Vars (Variables):** *An object containing CSS properties or GSAP-specific properties and their values.*
 - Written within curly braces `{}`.
 - Properties and values are given in key-value pairs (e.g., `x: 1000`).
- `gsap.to()`:** ➡
 - Animates an element from its current (initial) position/state to a specified (final) position/state.*
 - Example: Animate a box from where it is to `x: 1000`.

```
gsap.to("#box", {
  x: 1000,
  duration: 2,
  delay: 1
});
```
- `gsap.from()`:** ⬅
 - Animates an element from a specified (final) position/state to its current (initial) position/state.*
 - Useful for "reveal" animations where elements come into view from off-screen or from a different state.
 - Example: Animate a box from `x: 1000` to where it is.

```
gsap.from("#box", {
  x: 1000,
  duration: 2,
  delay: 1
});
```
- Common GSAP Properties (Vars):**
 - `x`, `y`: *Moves an element horizontally (X-axis) or vertically (Y-axis) in pixels.*
 - Example: `x: 1000` (moves 1000px right), `y: 500` (moves 500px down).
 - `duration`: *The total time (in seconds) the animation takes to complete.*
 - Example: `duration: 2` (animation lasts 2 seconds).
 - `delay`: *The time (in seconds) *before* the animation begins.*
 - Example: `delay: 1` (animation starts 1 second after the page loads or preceding animations finish).
 - `rotate`: *Rotates an element by a specified degree.*
 - Example: `rotate: 360` (rotates 360 degrees).
 - `scale`: *Resizes an element.*
 - Example: `scale: 0.5` (half size), `scale: 2` (double size).
 - `opacity`: *Adjusts the transparency of an element.*
 - Example: `opacity: 0` (fully transparent/invisible). When used with `from()`, it means starting invisible and fading to visible.
 - Using CSS Properties:**
 - Many standard CSS properties can be animated directly.
 - Camel Case Convention:** **For CSS properties with hyphens (e.g., `background-color`, `border-radius`), convert them to camel case (e.g., `backgroundColor`, `borderRadius`).*
 - Example: `backgroundColor: "blue"`, `borderRadius: "50%"`
 - String Values:** **Values that are words or include special characters (like `%`) must be enclosed in double quotes.**
 - Example: `"blue"`, `"50%"`. Numeric values like `1000` or `2` do not need quotes.

🌟 4. Advanced GSAP Features

- `stagger`:** ⏸
 - Used to animate multiple elements (e.g., all `h1` tags) sequentially, with a specified delay between each animation.*
 - Syntax: `stagger: 0.5` (each element animates 0.5 seconds after the previous one).
 - Negative values: `stagger: -1` (animates elements in reverse order).
 - Example:

```
gsap.from("h1", {
  y: 20,
  opacity: 0,
  stagger: 0.3
});
```
- `repeat`:** 🔄
 - Specifies how many times an animation should repeat after its initial run.*
 - `repeat: 1`: The animation plays once, then repeats once (total 2 runs).
 - `repeat: 2`: The animation plays once, then repeats twice (total 3 runs).
 - `repeat: -1`: **The animation repeats infinitely.**
 - Example: `repeat: -1`
- `yoyo`:** 🎱
 - When used with `repeat`, makes the animation play forwards and then immediately backward. (Like a yo-yo toy).*
 - Syntax: `yoyo: true`
 - Requires `repeat` to be effective (e.g., `repeat: -1, yoyo: true`).
 - Example: An element goes right, then comes back left, then goes right, etc.

🕒 5. GSAP TimeLine

```
+-----+
| Timeline (A Train) |
| [Animation 1] -- [Animation 2] -- [Animation 3] |
+-----+
```

- **Problem with Separate Animations:**
 - When animating multiple elements sequentially, manually calculating and setting `delay` for each subsequent animation (based on the `duration` and `delay` of previous ones) becomes complex and error-prone.
 - This is an *asynchronous* approach, where each animation is managed independently.
- **Solution: `gsap.timeline()`:**
 - A powerful tool to orchestrate multiple GSAP animations, allowing them to run in a specific sequence or with precise overlaps without manual `delay` calculations.
 - This creates a *synchronous* flow, like a train where each bogie (animation) follows the previous one.

- **How to Use Timeline:**

1. **Create a Timeline Instance:**

```
let tl = gsap.timeline();
```

(You can name the variable anything, e.g., `myTimeline`, `tl`).

2. **Add Animations to the Timeline:**

- Instead of `gsap.to()` or `gsap.from()`, use `tl.to()` or `tl.from()`.
- Animations added to a timeline will run one after the other by default, in the order they are added.
- No need to manually calculate `delay` for sequential animations.

```
tl.to("#box1", { x: 1500, rotate: 360, duration: 1.5, delay: 1 });
tl.to("#box2", { x: 1500, backgroundColor: "yellow", duration: 1.5 }); // Starts after box1 finishes
tl.to("#box3", { x: 1500, scale: 0.5, borderRadius: "50%", duration: 1.5 }); // Starts after box2 finishes
```

- **Benefit:** Simplifies complex animation sequences, especially for dynamic layouts like navigation bars where elements need to appear in a specific order.

...